

## CMPE 322 Project 2 Report

Ahmet Meriç Kızıltas - 2022400225

### 1. Manual Pages Used

- **pthread\_create:** [https://man7.org/linux/man-pages/man3/pthread\\_create.3.html](https://man7.org/linux/man-pages/man3/pthread_create.3.html)
- **pthread\_join:** [https://man7.org/linux/man-pages/man3/pthread\\_join.3.html](https://man7.org/linux/man-pages/man3/pthread_join.3.html)
- **pthread\_exit:** [https://man7.org/linux/man-pages/man3/pthread\\_exit.3.html](https://man7.org/linux/man-pages/man3/pthread_exit.3.html)
- **pthread\_mutex\_init:** [https://man7.org/linux/man-pages/man3/pthread\\_mutex\\_init.3.html](https://man7.org/linux/man-pages/man3/pthread_mutex_init.3.html)
- **pthread\_mutex\_destroy:** [https://man7.org/linux/man-pages/man3/pthread\\_mutex\\_destroy.3.html](https://man7.org/linux/man-pages/man3/pthread_mutex_destroy.3.html)
- **pthread\_mutex\_trylock:** [https://man7.org/linux/man-pages/man3/pthread\\_mutex\\_lock.3.html](https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3.html)
- **pthread\_mutex\_unlock:** [https://man7.org/linux/man-pages/man3/pthread\\_mutex\\_unlock.3.html](https://man7.org/linux/man-pages/man3/pthread_mutex_unlock.3.html)
- **clock\_gettime:** [https://man7.org/linux/man-pages/man3/clock\\_gettime.3.html](https://man7.org/linux/man-pages/man3/clock_gettime.3.html)

### 2. Explanation of Functions

I implemented the following functions to achieve the project goals:

**array\_allocation:** In this function, I reserved the memory space for our three main arrays: the hash table itself, the list of items we want to insert, and the locks. I also initialized a separate mutex lock for every single slot in the table. This preparation is necessary before we start any operations.

**array\_deallocation:** This function cleans up everything after we are done. I first destroyed all the mutex locks to free system resources. Then, I freed the memory for the hash table, the entry list, and the lock array. This makes sure we do not have any memory leaks.

**sequential\_h\_1:** This function performs the insertion task using a single thread. It takes items one by one and uses a linear probing strategy. If a slot is full, it moves to the next one. I used "clock\_gettime" here to give every item a precise timestamp when it is placed.

**parallel\_h\_1:** This function manages the multi-threaded execution. First, it creates "t" threads. Each thread acts independently using the runner\_h1 logic. Inside the runner, the thread calculates an index and checks if the slot is empty before trying to lock it. This saves time. If the lock is acquired, I check the slot one last time to be safe, then insert the item and timestamp it.

**parallel\_h\_2:** This function sets up the threads for the Double Hashing approach. The runner logic runner\_h2 is more complex here to prevent deadlocks. A thread calculates the "n/k" specific locks it needs to verify its path. It tries to acquire them one by one. If it cannot get a lock immediately, it releases all the locks it already holds and starts over. This guarantees that threads never get stuck waiting for each other.

**speedup\_comparison\_h\_1:** This function measures the performance improvement. First, I run the sequential function and measure its duration using "clock\_gettime(CLOCK\_MONOTONIC)". Then, I clear the table and run the parallel function, measuring its duration the same way. Finally, I divide the sequential time by the parallel time to calculate the speedup ratio.

### 3. Comparison of h1 and h2

The h1 function is much faster than h2 because it only needs to grab one lock at a time, which reduces the waiting time for threads. On the other hand, h2 is safer for complex tables but slower because threads must grab many locks at once, and if they fail, they have to restart the whole process. Therefore, h1 works best for speed, while h2 ensures robust correctness in deadlock-prone situations.

### 4. Research Topics

I researched concurrency concepts including "Locking", "Deadlock Prevention", and "Livelock". I also studied looked Amdahl's Law to understand how much faster the program can theoretically get with four cores.

### 5. Chat History

<https://drive.google.com/file/d/1AtdW8rz7JxPe8s63amC0MwJZhyJnqqPZ/view?usp=sharing>